Adaptive Portfolio Management in Volatile Markets with Multiplex Attention Transformers and Deep Reinforcement Learning

Angela Mi[*], Pei Xue[†], Joanna Tan[‡]
10-423/623 Generative AI Course Project

1 May 2025

1 Introduction

Our project proposes an integrated deep learning architecture that combines a Transformer model with a Reinforcement Learning (RL) agent for dynamic stock portfolio management. The Transformer extracts predictive features from high-dimensional, time series financial data—enhanced with market sentiment derived via a pretrained FinBERT model—while the RL agent makes real-time portfolio decisions based on the stable, frozen outputs from the Transformer and additional live market and user portfolio data.

Financial markets exhibit complex, non-stationary behaviors driven by a confluence of technical, macroeconomic, and sentiment factors. Traditional forecasting models often struggle to capture long-range temporal dependencies or fail to translate those signals into profitable trading strategies, while standalone RL agents lack reliable forward-looking features and can overfit noisy price data. By uniting a multiplex-attention Transformer trained on multimodal time-series and sentiment inputs with a DRL agent that operates on its frozen latent embeddings, we seek to overcome these limitations.

2 Dataset / Task

Dataset: The Transformer dataset $\mathcal{D}_T$ integrates three modalities: daily stock OHLC price series and derived technical indicators (volume, moving averages, MACD) sourced from Yahoo Finance; sentiment embeddings computed via a pretrained FinBERT model from curated news scraped from Google News, sector-level market indices and key macroeconomic indicators (federal funds rate, S&P 500, Dow Jones).

The dataset comprises daily price indicators for the 30 constituents of the Dow Jones Industrial Average (DJIA30) on NYSE trading days between January 2009 and April 2025. Observations are partitioned into a training set spanning Janurary 2009 to June 2020, and a validation/backtesting dataset from July 2020 to March 2025. Non-trading days (weekends and exchange holidays) were omitted.

The RL dataset $\mathcal{D}_{RL}$ utilizes the standardized market environment specifications from the FinRL-Meta benchmark suite[Liu et al., 2022b].

Task statement: The model aims to make portfolio decisions (Buy, Sell, Hold, and determine trade volume) based on real-time market environments, using the predicted future stock price trends from the Transformer output to inform the RL agent's state space.

Evaluation Metrics: As this work extends established financial forecasting and trading benchmarks, the Transformer's forecasting accuracy is assessed by mean squared error, compared against a LSTM model [Gülmez, 2023]. Portfolio performance is measured by cumulative return on investment (ROI), annualized Sharpe ratio, and maximum drawdown, in accordance with the FinRL-Meta evaluation protocol [Liu et al., 2022a].

3 Related Work

Recent advances in generative AI and deep reinforcement learning (DRL) have sparked growing interest in developing intelligent trading agents that combine structured market data with unstructured information sources. Large language models (LLMs) such as FinFinBERT and GPT-based architectures have demonstrated strong capabilities in extracting market-relevant signals from financial news, earnings reports, and social media, enhancing decision-making in trading environments [Yang et al., 2020, Xue et al., 2024]. Li et al. [2023b] proposed TradingGPT, an architecture that integrates LLMs with DRL to enable agents to reason over text-based market information and generate adaptive trading policies. This hybrid approach shows promise in improving generalization and sample efficiency by allowing agents to lever-

[*]chenjinm@andrew.cmu.edu
[†]pxue@andrew.cmu.edu
[‡]joannata@andrew.cmu.edu

age contextual financial knowledge. Prior work by [Ding et al., 2020] enhanced the Transformer with multi-scale Gaussian priors and hierarchical attention masks to capture intra-day and intra-week patterns, and [Li et al., 2023a] proposed MASTER, which applies a market-driven gating mechanism and alternates intra-stock and inter-stock attention to model momentary and cross-time correlations.

In parallel, generative models such as GANs and diffusion models have been used to simulate realistic market scenarios and augment training data, helping DRL agents learn under a broader set of conditions [Wiese et al., 2020, Owen, 2025]. Liu et al. [2024] developed an open-source infrastructure for building realistic, reproducible, and scalable financial RL environments. Under this framework, agents gain access to dynamic financial datasets, hundreds of standardized Gym-style environments, and reproducible benchmarks for portfolio trading. Together, these efforts point toward a new class of foundation model-powered trading agents that blend symbolic reasoning with continuous action learning for more robust and intelligent portfolio management.

4 Approach

4.1 Baseline Model

The baseline model for portfolio construction consists of a layered architecture with the RL agent and environment layers. The agent layer uses DRL algorithms like PPO, A2C, and DDPG, trained on historical market data to optimize returns. The environment layer simulates market conditions using OHLCV data and technical indicators. The model follows a training-testing-trading pipeline, which involves training on historical data, backtesting, and real-time deployment with live trading APIs.

The baseline model exhibits several key limitations. First, it relies heavily on traditional DRL algorithms such as PPO, A2C, and DDPG, which often struggle to capture long-range dependencies in high-dimensional, time-series financial data. These algorithms are limited in their ability to understand complex patterns that evolve over extended periods, such as macroeconomic trends or market sentiment shifts. Second, the baseline model primarily uses OHLCV data and technical indicators for market condition simulation, which fails to incorporate external signals like news sentiment or broader macroeconomic factors. This results in suboptimal performance when predicting stock price movements based solely on historical raw price data, which is inherently noisy and volatile.

In contrast, our proposed approach enhances the baseline model by introducing a Transformer model to predict future stock prices and trends. This Transformer, unlike traditional recurrent models, leverages self-attention mechanisms to simultaneously capture features from different parts of the input sequence. It also incorporates market sentiment from a pre-trained FinBERT model and technical indicators, replacing noisy historical stock prices with more robust inputs. Additionally, our approach integrates the output of the Transformer as a fixed representation of market conditions for an RL agent, enabling more informed portfolio decisions. This two-component pipeline—Transformer for price prediction and RL agent for portfolio management—addresses the baseline's shortcomings, providing more accurate and dynamic predictions that can adapt to evolving market conditions.

4.2 Improvement

The proposed approach is divided into two major components that are tightly integrated into a pipelined process: a Transformer model trained to predict continuous variables related to future stock prices and trends, and a Reinforcement Learning (RL) agent trained to make portfolio decisions based on the Transformer's output as a fixed representation of market conditions in addition to the user's portfolio data.

Instead of traditional recurrent architectures such as RNNs or LSTMs, the Transformer is chosen for its ability to extract features simultaneously from different parts of the input sequence and better understand the high-dimensional input data that span over long time series ranges using self-attention mechanisms. The input embedding consists of financial data along with daily market sentiment data passed in from a sentiment analysis model with scraped stock news from Google News as inputs. In particular, the sentiment model ingests each news text's scores from FinBERT, VADER, and the Loughran–McDonald lexicon and uses majority voting to output a daily label of positive, negative, or neutral. To mitigate issues of inherent noise present in raw stock prices data, technical indicators that hint trends in stock prices are embedded into the Transformer's input in replacement of historical raw stock prices. The model is then trained in a regression framework using Mean Square Error (SmoothL1) as the loss function, with its weights frozen once trained so that its predictions serve as stable inputs to the RL agent.

The RL model builds on the FinRL-Meta framework [Liu et al., 2022a], leveraging dynamic data sets, with live real-world data from 30+ sources including Yahoo and WRDS and user-defined portfolio data. The state

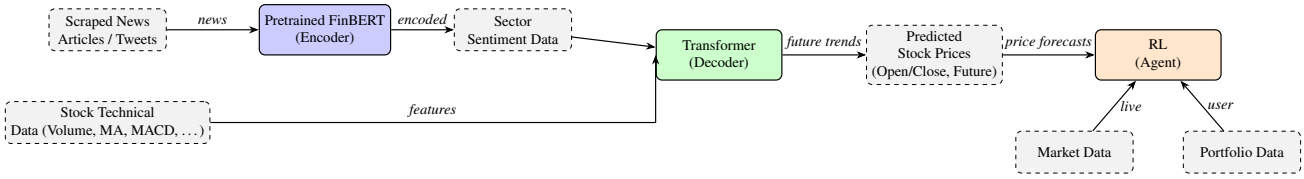

Figure 1: Proposed Pipeline: A Transformer-based feature extractor (Decoder) informed by sector sentiment (via a pretrained FinBERT Encoder) and technical data. Its future price predictions serve as inputs to an RL agent, which also ingests live market and user portfolio data to make portfolio decisions.

space consists of the current market conditions of the stock along with the Transformer's price predictions, leveraging the Transformer's ability to capture the future trajectory of stock prices, thereby influencing the timing and scale of trading actions.

4.2.1 Data Pre-processing and Sentiment Score Calculation

All market and sentiment inputs are first aligned to the official trading calendar and any gaps imputed by carrying forward the last observation. Technical indicators—such as moving averages, RSI, MACD, Bollinger Bands and volume—are computed for each symbol on each day, producing a continuous feature vector.

Sentiment is obtained by majority voting across FinBERT, VADER, and the Loughran–McDonald lexicon's preliminary scores. The headline label ℓ_h is the majority vote among $\{L_h^{\mathrm{FB}},\ L_h^{\mathrm{VD}},\ L_h^{\mathrm{LM}}\}$, and the daily sentiment score is

$$s_t = \frac{1}{N_t} \sum_{h=1}^{N_t} \ell_h.$$

To remove non-stationarity and avoid look-ahead bias, every feature X_t (technical, sentiment or macroeconomic) is rescaled to $[0, 1]$ via a Min–Max transform fitted exclusively on the training period:

$$x_t^{\mathrm{scaled}} = \frac{x_t - \min_{u \in \mathcal{T}_{\mathrm{train}}} x_u}{\max_{u \in \mathcal{T}_{\mathrm{train}}} x_u - \min_{u \in \mathcal{T}_{\mathrm{train}}} x_u}\,.$$

The fitted bounds are then applied unchanged to validation and test splits, preserving extreme-move amplitudes without leaking future information.

4.2.2 Transformer for Stock Price Prediction

Transformers are transduction models originally developed for NLP [Vaswani et al., 2023] but increasingly applied in financial time series prediction [Caron and Müller, 2020, Cong et al., 2020, Ma et al., 2023]. The popularity of Transformers stems from their ability to capture long-range dependencies via self and cross-attention.

Specifically, we implement both a vanilla transformer with multiplex attention and a Temporal Fusion Transformer (TFT) [Lim et al., 2020] with multiplex attention. Transformer specializes in time-series forecasting, while multiplex attention captures information from different sources of data.

Temporal Fusion Transformer (TFT) We employ the Temporal Fusion Transformer (TFT) as the core forecasting module due to its ability to capture both short-term and long-range temporal dependencies, while maintaining interpretability and accommodating diverse input types. TFT is designed specifically for multi-horizon time series forecasting, and supports dynamic feature selection, temporal attention, and probabilistic forecasting.

The TFT architecture consists of the several key components. First, to model short-term sequential dependencies, TFT includes an LSTM encoder-decoder that processes temporally ordered historical and future inputs. The outputs are enriched by static context vectors derived earlier. TFT then applies multi-head attention across the decoder's hidden states to identify salient past time steps that influence each forecasted output. This mechanism provides temporal interpretability while capturing long-range dependencies. Moreover, Gated Residual Networks (GRNs) form the building blocks of most layers in TFT. Each GRN contains a nonlinear transformation block followed by a gating mechanism and residual connection, enabling efficient gradient flow and adaptive information control. After the attention layer, position-wise GRNs further process the attention-enhanced features. Skip connections from earlier layers ensure stable learning by preserving low-level representations. Finally, instead of producing a single point estimate, TFT outputs quantile forecasts (typically 0.1, 0.5, 0.9), enabling a distributional view of future values. This supports downstream risk-sensitive decision-making. As such, TFT is particularly well-suited for financial forecasting tasks, where long-range interactions, mixed input types, and uncertainty quantification are essential. Figure 2 depicts the TFT architecture.

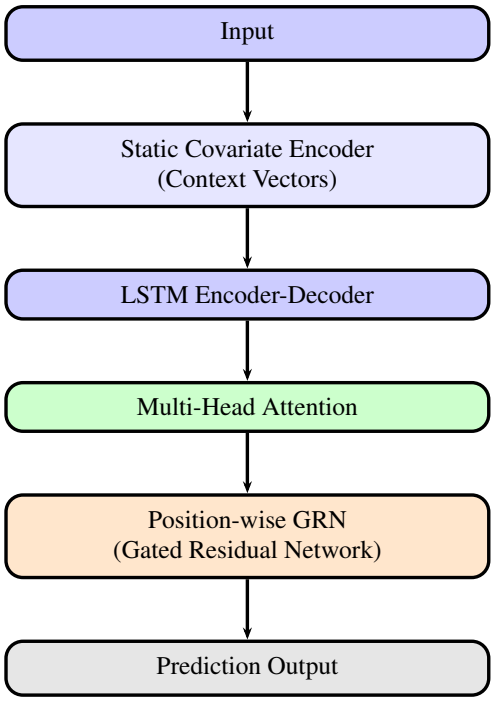

Figure 2: Temporal Fusion Transformer (TFT) Architecture: Static covariates are encoded into context vectors influencing the variable selection and LSTM modules. Past and future inputs are processed through separate variable selection networks and LSTM blocks, followed by interpretable multi-head attention and final position-wise GRNs to produce predictions.

Multiplex Attention For stock price prediction, we propose the use of multiplex attention [Xu et al., 2023] in Transformers to attend to multiple data sources, specifically historical stock technical data, news sentiment analysis, and macroeconomic data. The input data is normalized within each category to form the input embeddings $X_{stock}, X_{sent}, X_{macro}$. The news sentiment input X_{sent} is passed from the pre-trained encoder (FinBERT). Then, for each input $X_{stock}, X_{sent}, X_{macro}$ respectively, we pass the sequence through a decoder, calculating self-attention scores for each X_i via standard procedures:

$$\text{Attention}(Q, K, V) = \text{softmax}(\frac{Q_i K_i^T}{\sqrt{d}})V_i$$

$$Q_i = X_i W_{i,q}, K_i = X_i W_{i,k}, V_i = X_i W_{i,v}$$

Finally, we concatenate the attention scores from each input to produce output attention scores:

$$\text{Multi}(Q, K, V) = \text{Concat}(A_{stock}, A_{sent}, A_{macro})W_{out}$$

Using multiplex attention allows the model to incorporate both micro and macroeconomic information, resulting in more robust stock price predictions.

Multi-Step Sequence Window To capture both short- and long-term temporal dynamics, we structure the input as overlapping windows of length T, where each window $\{X_{t-T+1}, \ldots, X_t\}$ encodes the past T time steps of technical, sentiment, and macro features. A causal mask ensures that at each decoding step the model attends only to preceding positions, enabling the Transformer to learn autoregressive dependencies over multiple scales. During training, we predict the next H steps of price trajectories $\{\hat{y}_{t+1}, \ldots, \hat{y}_{t+H}\}$ and optimize with a combined SmoothL1 loss over the horizon. Here, we choose $T = 30$ and $H = 1$ for our model based on hyperparameter tuning on our validation splits.

Training and Prediction Model parameters are optimized using the AdamW optimizer with a linear warmup followed by cosine decay of the learning rate, minimizing the multi-step L1Smooth loss over the forecast horizon:

$$\mathcal{L} = \frac{1}{N}\frac{1}{H}\sum_{t=1}^{N}\sum_{h=1}^{H} \ell_{\text{L1Smooth}}(\hat{y}_{t+h} - y_{t+h}),$$

where the L1Smooth (Huber) loss is defined as

$$\ell_{\text{L1Smooth}}(\delta) = \begin{cases} \frac{\delta^2}{2\beta}, & |\delta| \le \beta, \\ |\delta| - \frac{1}{2}\beta, & |\delta| > \beta, \end{cases}$$

with threshold β set to 1.0. Training runs for a maximum of 100 epochs but employs early stopping—monitoring validation loss and terminating training if no improvement is observed for 10 consecutive epochs—to prevent overfitting and preserve the parameter state at optimal validation performance. At inference, the Transformer consumes input windows of shape $(B, T, 3d)$ and produces H-step forecasts

$$[\hat{y}_{t+1}, \ldots, \hat{y}_{t+H}] = f_\theta(\mathbf{X}_t),$$

using the weights selected by early stopping on held-out data.

Finetuning To better adapt the Temporal Fusion Transformer to the specific characteristics of our test period, we performed continuing training for ten epochs on the 2024-2025 validation data split. This targeted refinement aligns the model's parameters with the most recent market dynamics and yields a more faithful assessment of out-of-sample performance. In future work, we plan to extend fine-tuning to larger and more diverse data windows to further enhance robustness across varying market regimes.

4.2.3 Deep Reinforcement Learning for Portfolio Management

We implement a **Deep Reinforcement Learning (DRL)** framework to train an **intelligent trading agent**

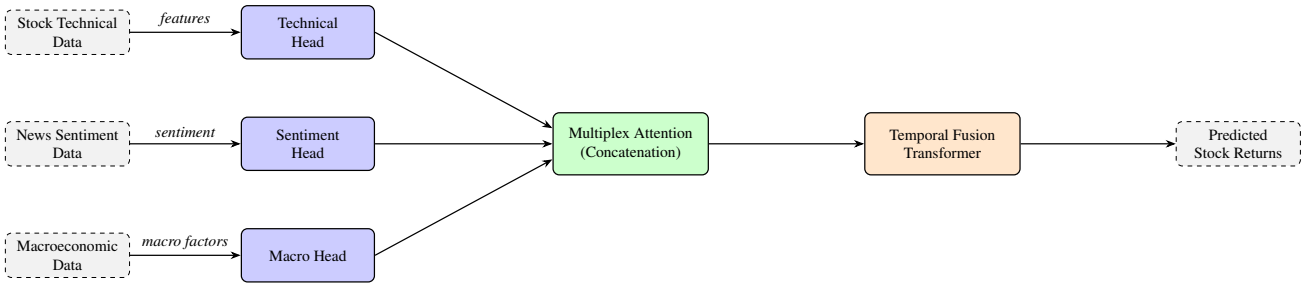

Figure 3: Proposed multiplex attention: Each input modality - stock technical features, news sentiment, and macroeconomic indicators — is processed by a specialized attention head.

capable of making sequential portfolio decisions in a dynamic financial environment. The trading task is modeled as a Markov Decision Process (MDP), where the agent observes a state s_t, takes an action a_t, receives a reward r_t, and transitions to the next state s_{t+1}.

The state includes current prices, technical indicators, portfolio holdings, and risk signals. Actions represent portfolio weights across n assets, and the reward is typically the change in portfolio value.

The agent learns a policy $\pi_\theta(s_t)$, parameterized by a neural network, that maximizes the expected cumulative return.

We leverage DRL algorithms such as PPO, SAC, and TD3, which balance reward maximization with stability and risk awareness. Once trained, the agent deploys the learned policy to make data-driven, adaptive trading decisions that respond to evolving market conditions.

Extended State Space Building on the FinRL state definition [Liu et al., 2021], we augment the state with the Transformer's forecasts:

$$s_t = \big(b_t,\ \mathbf{h}_t,\ o_t,\ \ell_t,\ p_t,\ v_t,\ M_t,\ \mathrm{RSI}_t,\ \hat{Y}_t\big),$$

where all numerical inputs are standardized per asset to zero mean and unit variance, b_t is cash, $\mathbf{h}_t$ holdings, (o_t, ℓ_t, p_t, v_t) prices and volumes, M_t, RSI_t technical indicators, and $\hat{Y}_t = [\hat{y}_{t+1}, \ldots, \hat{y}_{t+H}]$ the H-step log-price forecasts.

Action and Reward Actions $a_t \in \mathbb{R}^n$ specify portfolio weight adjustments across n assets. The immediate reward is the portfolio value (PV) change

$$r_t = \mathrm{PV}_t - \mathrm{PV}_{t-1}, \qquad r'_t = r_t - \lambda\,\mathrm{Volatility}_t,$$

Additionally, we can embed risk penalties directly into the reward where λ regulates risk aversion.

Objective and Algorithms The agent learns a policy $\pi_\theta(s_t)$ to maximize the expected cumulative return:

$$\max_\theta \mathbb{E}_{\pi_\theta}\Big[\sum_{t=0}^{T} \gamma^t\, r_t\Big].$$

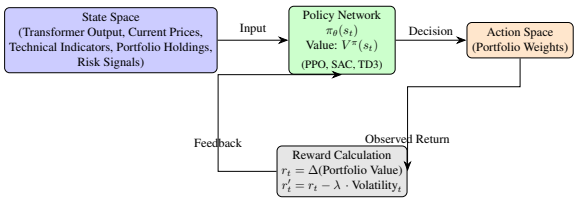

Figure 4: DRL Agent: The agent receives a composite state, processes it via a policy network.

We employ PPO, SAC and TD3 for stable, risk-aware learning. Including $\hat{Y}_t$ equips the agent with forward-looking signals, improving decision quality under volatile conditions.

5 Experiments

5.1 Baseline Model: Stock Prediction

To evaluate the Transformer-based stock predictor, we compare our model with pretrained LSTM-based models [Gülmez, 2023] on error in prediction of DJIA 30 stocks' prices from years 2018 to 2023. Specifically, Gülmez [2023] trains an LSTM1D, LSTM2D, LSTM3D, ANN, LSTM-GA, and optimized LSTM with ARO (LSTM-ARO). The MSE, MAE, MAPE, and R2 scores of the baseline model are compared against our Transformer implementations in Table 1.

5.2 Baseline Model: Portfolio Construction

5.2.1 Baseline experiments and Trading Performance Metrics

As baselines for our study, replicating works from FinRL [Liu et al., 2021], we train five deep reinforcement-learning agents—A2C, DDPG, PPO, TD3 and SAC—on historical daily OHLCV and technical-indicator data of the 30 DJIA constituents from January 1, 2009 to June 30, 2020, and evaluate (backtest) their performance over the out-of-sample period from July 1, 2020 to March 30, 2025. Each agent starts with an initial capital of $1 000 000 and learns to

trade by observing market states and receiving portfolio returns as rewards. In parallel, we construct a classical mean–variance portfolio by estimating expected returns μ and covariance matrix Σ on the same training data, then solving

$$\max_{w} \frac{w^\top(\mu - r_f)}{\sqrt{w^\top \Sigma\, w}} \quad \text{s.t.} \quad w_i \in [0, 0.5],\ \sum_i w_i = 1,$$

to maximize the Sharpe ratio under weight bounds, and we include a passive benchmark that buys and holds a DJIA index ETF over the test period.

Backtesting is carried out via an automated framework that simulates daily portfolio rebalancing at closing prices, assumes zero transaction costs and slippage, and uses a zero risk-free rate. Starting from the initial capital, each strategy's daily actions determine portfolio weights, the resulting portfolio value is recorded each day, and this time series is used to compute all performance metrics.

We assess all strategies using the following trading metrics: **final portfolio value** V_{final}, **cumulative return** $(V_{\text{final}} - V_{\text{initial}})/V_{\text{initial}}$, **annualized return** : geometric mean of daily returns scaled to one year, **annualized volatilit**: standard deviation annualized return, **Sharpe ratio** $(R_p - r_f)/\sigma_p$, and **maximum drawdown**: largest peak-to-trough decline.

Table 2 and Figure 5 summarizes the results. All DRL strategies and the MVO method outperformed the DJIA benchmark in terms of final portfolio value and annualized return. DDPG and SAC achieved final values of \$1.97M and \$1.92M, with annualized returns of 15.45% and 14.81%, respectively, compared to DJIA's \$1.62M and 10.69%. MVO achieved the highest final value (\$2.04M) and return (16.24%), suggesting that both learning-based and optimization-based approaches can potentially enhance investment outcomes over passive index tracking.

When comparing risk-adjusted performance, DDPG obtained the highest Sharpe ratio (0.99), indicating a favorable balance between return and volatility. TD3 and SAC also demonstrated competitive Sharpe ratios (0.83 and 0.88, respectively), though with varying levels of risk as measured by annualized standard deviation and maximum drawdown. MVO, while strong in overall return, exhibited relatively higher volatility and drawdown, highlighting trade-offs between performance and stability across different strategies. Despite these results, the strategies still show room for improvement, particularly in controlling downside risk and reducing return volatility. Some DRL models experienced relatively high drawdowns (e.g., SAC at –28.50%), and the MVO approach also carried considerable risk. Future work may explore model refinements or hybrid strategies that better balance profitability with robustness under varying market conditions.

5.3 Results

5.3.1 Transformer Forecast Results

Table 1: Transformer forecasting performance averaged over DJIA-30 test split. Lower is better for MSE/MAE/MAPE; higher is better for R^2.

Model	MSE	MAE	MAPE	$\mathbf{R^2}$
LSTM-ARO	23.31	3.24	2.00%	0.912
LSTM-GA	23.56	3.28	2.04%	0.911
LSTM1D	118.27	6.64	4.06%	0.754
LSTM2D	139.77	7.15	4.30%	0.715
LSTM3D	106.77	6.88	4.52%	0.683
ANN	71.29	6.00	3.86%	0.715
Multiplex Transformer	**16.52**	**2.32**	**1.53%**	**0.979**
TFT Transformer	**10.54**	**1.99**	**1.27%**	**0.971**
TFT + Finetuning	**8.28**	**1.89**	**1.32%**	**0.969**

Table 1 includes results of our Multiplex Transformer and TFT Transformer (with and without fine-tuning) alongside all LSTM variants from the baseline. Metrics include MSE, MAE, MAPE and R^2, demonstrating the incremental gains from multiplex fusion and fine-tuning. All forecasting metrics are computed over the DJIA-30 test period from July 1, 2020 through March 31, 2024. For comparison, the baseline LSTM models were evaluated on the years 2018 – 2023. We use this test window for three reasons: (1) high-quality sentiment and macro covariates only stabilize in mid-2020, (2) starting after our own training cutoff (June 2020) prevents look-ahead leakage, and (3) the July 2020–March 2024 span captures distinct market regimes—including the post-COVID rebound, 2022 tightening drawdowns, and the 2023–2024 stabilization—while extending several months beyond the baseline's endpoint to demonstrate ongoing robustness. The results are obtained by averaging over the tickers over the test time span, using a rolling forecast with a lookback window of 30 days and prediction window of 1 day. See Appendix A for the full set of one-day-ahead rolling-forecast plots (Figure 6).

The Multiplex Transformer reduces the baseline LSTM-ARO MSE by 29% (from 23.31 to 16.52) and lowers MAE by 28% (from 3.24 to 2.32), while achieving a MAPE of 1.53% and an R^2 of 0.979. These gains arise because multiplex fusion integrates both recent ("short-term") signals such as high-frequency technical indicators and longer-term macroeconomic and sentiment covariates into a unified self-attention framework.

Adopting the Temporal Fusion Transformer (TFT) architecture yields a further 37% reduction in MSE (to 10.54) and a 14% reduction in MAE (to 1.99) relative to the Multiplex model. The TFT architecture dynamically weigh covariates by relevance at each prediction step. This mechanism not only captures short bursts of market momentum but also preserves stable long-term trends, which explains the drop in MAPE to 1.27% and the strong R^2 of 0.971 despite heightened market turbulence.

Finally, fine-tuning the TFT under a SmoothL1 (Huber) loss drives MSE down to 8.28 and MAE to 1.89—improvements of 20% and 5% over the untuned TFT, respectively. While the R^2 dips slightly to 0.969, this reflects Huber's robustness to extreme price jumps and outliers, which are common in crisis events. Overall, the fine-tuned TFT delivers the lowest tail-error risk and the most stable one-day-ahead forecasts throughout July 2020–March 2024, confirming that combining advanced fusion yields state-of-the-art predictive performance.

5.3.2 DRL Trade Results

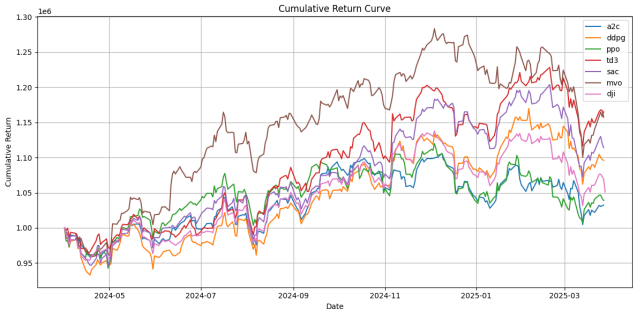

Figure 5: Performance of stock trading of Transformer-enhanced DRL Agent

Table 2: Comparison of Baseline and Transformer-Enhanced Stock-Trading Performance

Strategy	Ann. Return	Ann. Std	Sharpe Ratio	Max Drawdown
A2C	11.70% / **16.81%**	16.09% / **16.74%**	0.73 / **1**	–14.55% / **–25.25%**
DDPG	15.45% / **18.29%**	15.68% / **15.60%**	0.99 / **1.17**	–19.69% / **–15.86%**
PPO	12.10% / **16.01%**	15.15% / **15.75%**	0.80 / **1.02**	–22.24% / **–16.18%**
TD3	12.36% / **12.73%**	14.81% / **14.38%**	0.83 / **0.89**	–17.85% / **–19.81%**
SAC	14.81% / **17.07%**	16.83% / **18.22%**	0.88 / **0.94**	–28.50% / **–27.05%**
MVO	16.24% / **16.24%**	19.67% / **19.67%**	0.83 / **0.83**	–25.47% / **–25.47%**
DJIA	10.69% / **10.69%**	14.45% / **14.45%**	0.74 / **0.74**	–21.94% / **–21.94%**

Table 3: Model Hyperparameter Configurations After Fine-Tuning

Algorithm	n_steps	batch_size	buffer_size	learning_rate	ent_coef / tau	learning_starts
A2C	64	—	—	0.0001	0.0 (ent_coef)	—
DDPG	—	64	100 000	0.001	0.005 (tau)	—
PPO	2048	128	—	0.00025	0.01	—
TD3	—	100	1 000 000	0.001	—	—
SAC	—	128	100 000	0.0001	auto_0.1	100

Building on the established baselines, we implemented a Transformer-enhanced trading agent by integrating Transformer-based encoding into the DRL pipeline. Figure 5 displays the cumulative return curves for all baseline strategies over the test trading period. The updated performance results are summarized in Table 2, which compares each baseline model's original metrics with those of its Transformer-enhanced counterpart. Across all five DRL algorithms, the Transformer variant consistently achieved higher Sharpe ratios and annualized returns. For instance, A2C improved from a Sharpe ratio of 0.73 to 1.00 and from an annualized return of 11.70% to 16.81%, while DDPG increased its Sharpe ratio from 0.99 to 1.17 and its return from 15.45% to 18.29%. Predicted stock prices were incorporated into the state input for the central segment of the trading period, enhancing the agent's ability to anticipate trends and allocate capital more effectively. Although only a subset of predicted stock prices was used during training and trading due to computational constraints, the improved pipeline still led to consistently stronger performance across all evaluated metrics.

These results highlight that Transformer-enhanced models offer robust gains not only in profitability but also in risk-adjusted performance. Sharpe ratios improved across all DRL agents, often exceeding 1.0, indicating more stable and efficient reward generation relative to volatility. Furthermore, in several cases, such as A2C and PPO, the enhanced models outperformed the mean–variance optimizer (MVO) and the market benchmark (DJIA), suggesting that the Transformer architecture improves both learning capacity and generalization in dynamic financial environments. The accompanying hyperparameter table (Table 3) shows the fine-tuned configurations that contributed to these gains, particularly for A2C and DDPG. Together, these findings demonstrate the effectiveness of our Transformer-integrated reinforcement learning framework in improving quantitative trading strategies.

5.4 Computational Cost

All transformers are trained on two NVIDIA T4 GPUs. Training one epoch takes roughly 3 minutes and reaching convergence takes slightly over 2 hours. The batched inference on the entire validation dataset takes 2 minutes.

All DRL models were trained on a single NVIDIA T4 GPU using Google Colab. Training 10,000 timesteps took approximately 30–45 minutes per model, depending on the algorithm. Inference and backtesting for each trained agent completed in under 2 minutes.

6 Code Overview

The code can be found in our Github repository https://github.com/chenjinm/Trading-Bot.

6.1 Data Processing

For the data processing part, we wrote and uploaded the data collection code in the following files:

- `technical_data.ipynb`
- `sentiment_analysis_data.ipynb`
- `macro_data.ipynb`

These notebooks handle different modalities of the stock input data used for both the Transformer and RL models and output concatenated CSV files for the selected batch of stocks. In particular, we developed a custom sentiment scoring module that performs majority voting over NLTK VADER, ProsusAI FinBERT, and the Loughran–McDonald lexicon. To expedite scraping of long-range news data, we parallelized the RSS fetch for tickers on Colab's T4 GPUs.

6.2 Transformer Implementation

The core Transformer code is written from scratch and uploaded as `transformer.ipynb`, which implements the TFT and vanilla Transformer models in PyTorch. The main components we implemented include:

- **Custom PyTorch DataLoader**: Reads preprocessed CSVs of technical indicators, sentiment embeddings, and macro features; applies Min-Max scaling and z-score normalization per feature; constructs overlapping windows of length T with horizon H; and batches them as tensors of shape $(B, T, 3d)$
- **Model Architectures**: Three variants:
  Multiplex Transformer (from scratch): A custom PyTorch decoder implemented with multiplex self-attention, positional encodings, and feedforward layers for autoregressive multi-step forecasting.
  Temporal Fusion Transformer (Darts TFT): Utilizes the Darts library's `TFTModel` implementation, including static covariate encoders, variable selection networks, LSTM encoder–decoder submodules, and interpretable attention blocks [Lim et al., 2020].
  Multiplex Attention Module (from scratch): Computes separate self-attention heads for stock technical, sentiment, and macro inputs, concatenates the attention outputs, and projects them for downstream Transformer layers.
  The code for each component can be found respectively under each text header in the file.
- **Training Loop**: Utilizes AdamW with linear warmup and cosine decay, SmoothL1 (Huber) loss aggregated over the H-step horizon, gradient clipping (norm 1.0), and early stopping based on validation loss. Loss curves and horizon-error plots over all tickers are generated with Matplotlib.
- **Evaluation**: Computes MSE, MAE, MAPE, and R^2 on held-out test splits.

6.3 RL Implementation

In the submitted file drl_implementation.ipynb, Pei implemented the full training, evaluation, and comparison pipeline for deep reinforcement learning-based stock trading strategies, including both baseline models and the improved Transformer-enhanced version. This notebook contains code for training and fine-tuning five DRL algorithms—A2C, DDPG, PPO, TD3, and SAC—using the FinRL framework. For A2C and DDPG, we performed extensive hyperparameter tuning across multiple configurations to identify the best-performing models. The notebook also includes backtesting logic that evaluates each trained agent on a held-out trading period, computing key financial metrics such as annualized return, volatility, Sharpe ratio, and maximum drawdown. Additionally, the code includes implementation of the classical mean–variance optimizer (MVO) to serve as a baseline for performance comparison. This notebook provides a complete, self-contained workflow that demonstrates how the selected models perform under consistent evaluation conditions.

7 Timeline

To complete this project, we followed a structured workflow that included understanding prior work, implementing and modifying reinforcement learning agents, conducting experiments, and analyzing results. The process began with reviewing relevant literature, dataset documentation, and existing codebases. We then focused on training and fine-tuning five DRL algorithms, followed by integrating an improved model that incorporates Transformer-based predictions. This involved modifying code, writing new experiment scripts, and running both baseline and Transformer-based models under comparable settings. Finally, we compiled the results and prepared the final report. Table 4 provides an estimate of the hours spent on each of these activities. The breakdown reflects a balance

Table 4: Estimated Time Spent on Each Stage of the Project (all group members added together)

Project Stage	Hours Spent
Reading papers	6
Reading code documentation	5
Understanding existing implementation	6
Data collection and processing	28
Compiling/running baseline code	7
Modifying/writing DRL code	5
Modifying/writing Transformer code	20
Scripting DRL experiments	5
Scripting Transformer experiments	6
Running DRL experiments	40
Running Transformer experiments	55
Compiling results	5
Writing reports	25
Total	213

between development, experimentation, and evaluation phases, with additional effort allocated to the implementation and testing of the Transformer component.

8 Research Log

We encountered several key challenges during the process of training the Transformer models. First, an initial misconfiguration in our Min–Max scaler led to improperly scaled inputs, which caused out-of-control MSE values until we corrected both the scaling range and the subsequent z-score normalization by writing a custom class. Next, hyperparameter sweeps revealed that increasing model capacity—specifically embedding dimensions above 64, more than 2 encoder layers, or attention heads beyond 4—paradoxically degraded performance. We also experimented with various loss functions: early hybrid objectives combining MSE and binary cross-entropy to boost directional accuracy ultimately underperformed, and plain MSE produced noisy forecasts; in the end, SmoothL1 (Huber) loss yielded the most stable and lowest-horizon errors.

In parallel, our custom multiplex-attention Transformer was initially underperforming on the validation split, so we investigated alternative architectures and discovered the Temporal Fusion Transformer (TFT) implementation in the Darts library. Integrating the Darts TFT package produced substantially better multi-step forecasts, which led us to maintain two Transformer variants—our from-scratch multiplex model and the TFT model—to compare their respective strengths. Finally, a data collection oversight caused one DJIA30 ticker—CRM (Salesforce), which was only added to the index after our training cutoff (added to DJIA in 2020, so non-existent in the DJIA batch in our train data)—to skew our test metrics; once we identified and removed CRM as an outlier stock, our evaluation became far more reliable.

For the RL agent, another key challenge in this project was extending and fine-tuning the FinRL framework to support rigorous training and evaluation of multiple DRL algorithms. For A2C and DDPG, we implemented full-grid hyperparameter tuning to explore the impact of learning rate, batch size, buffer size, and step size on performance, while also ensuring consistent evaluation by aligning the trading periods and environment settings across models. Another significant effort involved modifying the DRL environment to incorporate predicted stock prices into the agent's state representation. This required not only expanding the state space to include predicted values alongside traditional features like current price, holdings, and cash, but also re-engineering several technical indicators to be computed using the predicted price series. These changes introduced a form of temporal foresight into the agent's observation space, allowing it to reason over both historical trends and model-informed expectations of future prices. The result was a more expressive and informative input representation that significantly enhanced the agent's ability to make proactive portfolio allocation decisions under uncertainty.

9 Conclusion

Our main findings show that embedding Transformer-generated predictions into the DRL state space can lead to substantial improvements in trading performance. Among the five algorithms evaluated, A2C and DDPG exhibited the most notable gains after fine-tuning, with Sharpe ratios rising from 0.73 to 1.00 and from 0.99 to 1.17, respectively. These improvements were achieved by optimizing key learning parameters and enriching the agent's input space through predicted prices. Financial indicators were recalculated using these predicted prices, allowing agents to learn from forward-looking signals. This enhancement led to better-informed decision making and outperformance relative to both default DRL baselines and traditional benchmarks.

Future work could explore further architectural improvements to DRL agents, particularly by incorporating Transformer layers directly into the policy or value networks. While in this project the Transformer's output was used externally to enrich the state space, an end-to-end approach that integrates attention mechanisms into the DRL architecture may improve the agent's ability to model sequential dependencies and long-term patterns. Such integration could be especially beneficial in environments with high temporal complexity and noise, such as financial markets.

Additionally, future research can investigate the use of adaptive state representations that evolve with market regimes or asset dynamics. Incorporating reinforcement learning techniques for automated hyperparameter optimization or agent selection could also reduce tuning costs and improve scalability across larger stock universes or multi-agent settings. These directions would build on the current pipeline while pushing the performance boundaries of DRL-based trading systems.

References

Matthew Caron and Oliver Müller. Hardening soft information: A transformer-based approach to forecasting stock return volatility. In *2020 IEEE International Conference on Big Data (Big Data)*, pages 4383–4391, 2020. doi: 10.1109/BigData50022.2020.9378134.

Lin William Cong, Ke Tang, Jingyuan Wang, and Yang Zhang. Deep sequence modeling: Development and applications in asset pricing. *The Journal of Financial Data Science*, 3(1):28–42, December 2020. ISSN 2640-3943. doi: 10.3905/jfds.2020.1.053. URL http://dx.doi.org/10.3905/jfds.2020.1.053.

Qianggang Ding, Sifan Wu, Hao Sun, Jiadong Guo, and Jian Guo. Hierarchical multi-scale gaussian transformer for stock movement prediction. In Christian Bessiere, editor, *Proceedings of the Twenty-Ninth International Joint Conference on Artificial Intelligence, IJCAI-20*, pages 4640–4646. International Joint Conferences on Artificial Intelligence Organization, 7 2020. doi: 10.24963/ijcai.2020/640. URL https://doi.org/10.24963/ijcai.2020/640. Special Track on AI in FinTech.

Burak Gülmez. Stock price prediction with optimized deep lstm network with artificial rabbits optimization algorithm. *Expert Systems with Applications*, 227:120346, 2023. ISSN 0957-4174. doi: https://doi.org/10.1016/j.eswa.2023.120346. URL https://www.sciencedirect.com/science/article/pii/S0957417423008485.

Tong Li, Zhaoyang Liu, Yanyan Shen, Xue Wang, Haokun Chen, and Sen Huang. Master: Market-guided stock transformer for stock price forecasting, 2023a. URL https://arxiv.org/abs/2312.15235.

Yang Li, Yangyang Yu, Haohang Li, Zhi Chen, and Khaldoun Khashanah. Tradinggpt: Multi-agent system with layered memory and distinct characters for enhanced financial trading performance. *arXiv preprint arXiv:2309.03736*, 2023b.

Bryan Lim, Sercan O. Arik, Nicolas Loeff, and Tomas Pfister. Temporal fusion transformers for interpretable multi-horizon time series forecasting. 2020. URL https://arxiv.org/abs/1912.09363.

Xiao-Yang Liu, Hongyang Yang, Jiechao Gao, and Christina Dan Wang. Finrl: Deep reinforcement learning framework to automate trading in quantitative finance. In *Proceedings of the second ACM international conference on AI in finance*, pages 1–9, 2021.

Xiao-Yang Liu, Ziyi Xia, Jingyang Rui, Jiechao Gao, Hongyang Yang, Ming Zhu, Christina Dan Wang, Zhaoran Wang, and Jian Guo. Finrl-meta: Market environments and benchmarks for data-driven financial reinforcement learning, 2022a. URL https://arxiv.org/abs/2211.03107.

Xiao-Yang Liu, Hongyang Yang, Qian Chen, Runjia Zhang, Liuqing Yang, Bowen Xiao, and Christina Dan Wang. Finrl: A deep reinforcement learning library for automated stock trading in quantitative finance, 2022b. URL https://arxiv.org/abs/2011.09607.

Xiao-Yang Liu, Ziyi Xia, Hongyang Yang, Jiechao Gao, Daochen Zha, Ming Zhu, Christina Dan Wang, Zhaoran Wang, and Jian Guo. Dynamic datasets and market environments for financial reinforcement learning. *Machine Learning - Springer Nature*, 2024.

Tian Ma, Wanwan Wang, and Yu Chen. Attention is all you need: An interpretable transformer-based asset allocation approach. *International Review of Financial Analysis*, 90:102876, 2023. ISSN 1057-5219. doi: https://doi.org/10.1016/j.irfa.2023.102876. URL https://www.sciencedirect.com/science/article/pii/S1057521923003927.

AE Owen. Ai-driven stress testing framework for credit portfolios using mcmc simulation. 2025.

Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. Attention is all you need. 2023. URL https://arxiv.org/abs/1706.03762.

Magnus Wiese, Robert Knobloch, Ralf Korn, and Peter Kretschmer. Quant gans: deep generation of financial time series. *Quantitative Finance*, 20(9):1419–1440, 2020.

Caosen Xu, Jingyuan Li, Bing Feng, and Baoli Lu. A financial time-series prediction model based on multiplex attention and linear transformer structure. *Applied Sciences*, 13(8), 2023. ISSN 2076-3417. doi: 10.3390/app13085175. URL https://www.mdpi.com/2076-3417/13/8/5175.

Zhiyi Xue, Liangguo Li, Senyue Tian, Xiaohong Chen, Pingping Li, Liangyu Chen, Tingting Jiang, and Min Zhang. Llm4fin: Fully automating llm-powered test case generation for fintech software acceptance testing. In *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis*, pages 1643–1655, 2024.

Yi Yang, Mark Christopher Siy Uy, and Allen Huang. Finbert: A pretrained language model for financial communications. *arXiv preprint arXiv:2006.08097*, 2020.

A All Stock-Prediction Plots

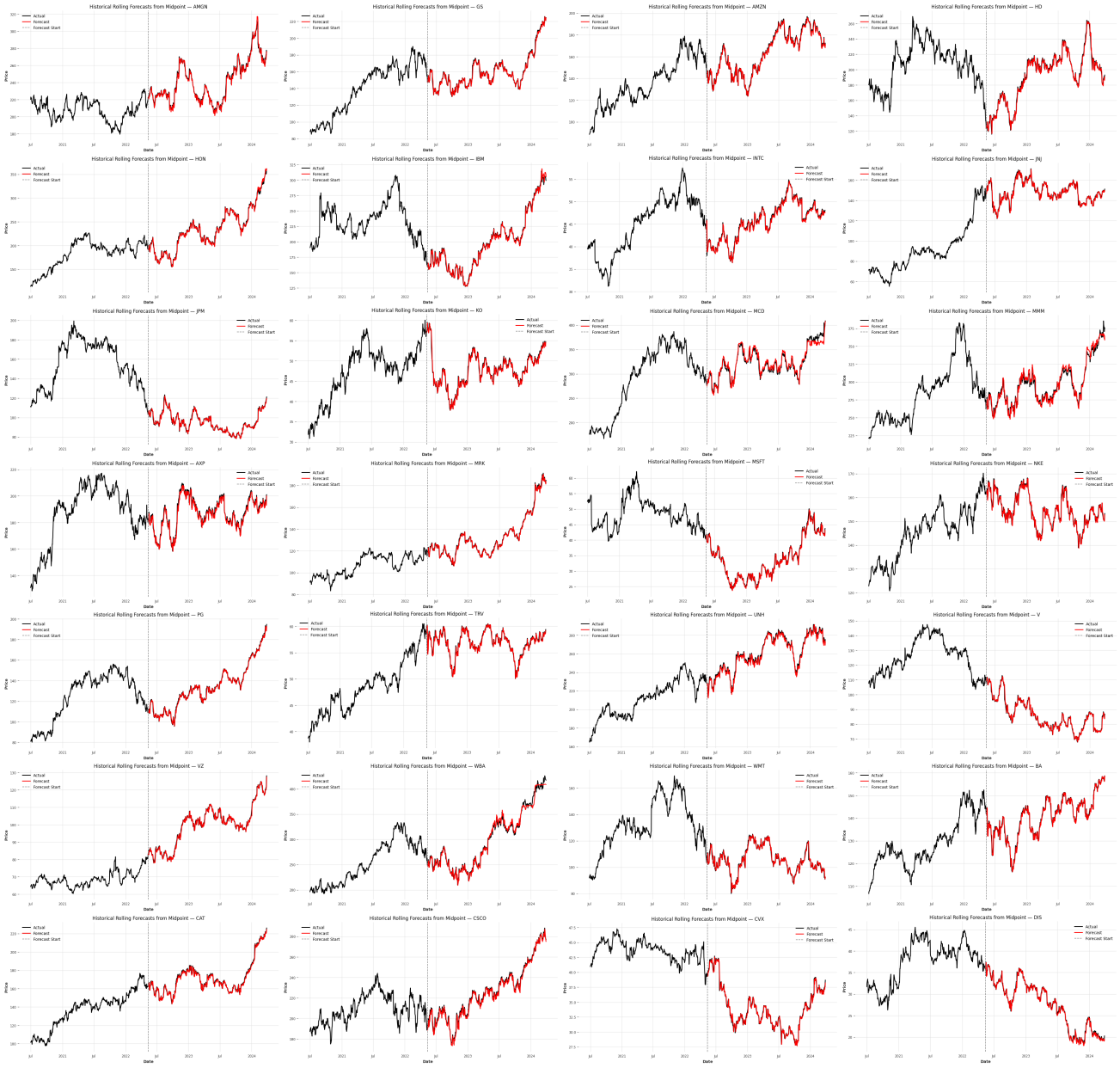

Figure 6: One-day-ahead rolling forecasts for all 28 DJIA constituents over July 2020–March 2024 using the TFT Transformer model, generated with a 30-day lookback and a 1-day prediction horizon. Constituents added to or removed from the DJIA30 index during this period are excluded.